

Keuze - welke prototype database

Algemene gegevens

Metadata	
Naam student	Bram Wieringa
Versienummer	1.1
Datum huidige versie	20-9-2026
Referenties	

1. Inleiding

Om snel te kunnen experimenteren met een deel van de aangeleverde dataset voor de projectopdracht, is een geschikte lokale database-omgeving nodig. Het doel is om deze dataset lokaal in te laden en klaar te zetten voor latere datatransformaties.

In dit document wordt op gestructureerde wijze onderzocht welke database-oplossing het beste aansluit bij de gestelde kaders, hoe de inrichting via scripting kan plaatsvinden en hoe deze integreert met dbt (data build tool).

2. Kaders & Randvoorwaarden

De database-oplossing en de bijbehorende pijplijn dienen te voldoen aan de volgende eisen:

1. **Native voor Linux:** Volledige ondersteuning en optimale performance op een Linux-omgeving.
2. **CLI & Scriptable:** Bediening via de Command Line Interface (CLI) heeft de voorkeur boven een GUI. De gehele opzet en data-ingestie moeten via scripts (zoals Shell, SQL of Python) te automatiseren zijn, zodat alle configuraties en stappen in versiebeheer (Git) vastgelegd kunnen worden.
3. **Ondersteuning voor `.csv.gz`:** Directe of efficiënte verwerking van gecomprimeerde CSV-bestanden zonder dat handmatige uitpakstappen nodig zijn.
4. **dbt-Integratie:** Naadloze samenwerking met dbt voor het uitvoeren van modulaire datatransformaties op de opgebouwde lokale dataset.
5. **Snel te installeren & makkelijk te verwijderen:** Omdat het om een snel experiment/onderzoek gaat, moet de database-engine direct in te richten zijn zonder ingewikkelde achtergrondservices of daemons. Na afloop moet de complete database eenvoudig te verwijderen zijn (bijvoorbeeld door het wissen van één enkel bestand of een tijdelijke map), zodat er geen rommel op het systeem achterblijft.

3. Onderzoeksmethodologie (ICT Research Methods)

Om tot een onderbouwde keuze en een werkende opstelling te komen, wordt gebruikgemaakt van het **DOT Framework** (Development-Oriented Triangulation) volgens *ICT Research Methods*. Dit framework bestaat uit drie complementaire onderzoeksmethoden, die elk vanuit een ander perspectief bijdragen aan de onderbouwing van de uiteindelijke oplossing:

- **Available Product Analysis (Library):**
 - *Doel:* Het vergelijken van geschikte lokale database-technologieën die voldoen aan de kaders uit Hoofdstuk 2 (bijv. DuckDB vs. ClickHouse Local vs. SQLite).
 - *Toepassing:* Bepalen welke engine de beste ingebouwde functionaliteit biedt voor het direct inlezen van gecomprimeerde bestanden (`.csv.gz`) en CLI-scripting.
- **Prototyping (Lab):**
 - *Doel:* Het opzetten van een minimale Proof of Concept (PoC).
 - *Toepassing:* Een CLI-script schrijven dat een lokaal `.csv.gz`-bestand inlaadt in de gekozen database en verifiëren dat dit herhaalbaar is via Git-gecontroleerde scripts.
- **Integration Test (Workshop / Lab):**
 - *Doel:* Het valideren van de keten met dbt.
 - *Toepassing:* De lokale database koppelen aan een dbt-profiel (`profiles.yml`) en controleren of een basistransformatie (`dbt run`) correct wordt uitgevoerd.

Dit document richt zich primair op de eerste methode, de **Available Product Analysis**: op basis van een systematische vergelijking van database-architecturen en -tools wordt onderbouwd welke oplossing het beste aansluit bij de gestelde kaders. De **Prototyping** en **Integration Test** volgen als praktische vervolgstappen op de hier gemaakte keuze en worden als concreet advies uitgewerkt in paragraaf 6.2.

4. Analyse van Database-architecturen

Welk type database-architectuur is het meest geschikt voor deze use-case?

Om een onderbouwde keuze te maken, zijn de verschillende database-technologieën ingedeeld op basis van hun specifieke type en architectuur. Elk type is getoetst aan de randvoorwaarden van een snel, lichtgewicht CLI-experiment op gecomprimeerde datasets (`.csv.gz`) met dbt-ondersteuning.

4.1 Overzicht en evaluatie van de types/architecturen

1. In-process OLAP (Online Analytical Processing):

- *Kenmerken:* Slaat gegevens kolomgeoriënteerd op en draait direct als bibliotheek/binary binnen het proces van de aanroepende applicatie of CLI-script. Geen achtergrondservice (daemon) nodig.
- *Geschiktheid voor use-case:* **Uitstekend** Volledig geoptimaliseerd voor analytische SQL-queries op grote bestanden. Kan `.csv.gz`-bestanden direct inlezen zonder

decompressie op schijf. Installatie en opruimen (wissen van 1 bestand) duren slechts seconden.

2. Columnar OLAP (Standalone / Server-based):

- *Kenmerken:* Eveneens kolomgeoriënteerd en gebouwd voor zware analyses, maar draait van oudsher als een dedicated server-daemon of cluster. Sommige moderne varianten bieden echter een speciale "local / in-process" CLI-modus.
- *Geschiktheid voor use-case:* **Goed.** Biedt extreem snelle query-performance op gecomprimeerde bestanden. De installatiefootprint en configuratie zijn wel zwaarder dan bij een pure in-process engine.

3. In-process OLTP (Online Transaction Processing):

- *Kenmerken:* Draait lokaal als een enkel bestand zonder achtergrondservice (zoals SQLite), maar is **rijgeoriënteerd** en ontworpen voor snelle, afzonderlijke schrijf- en leesacties (transacties).
- *Geschiktheid voor use-case:* **Matig.** Weliswaar zeer snel te installeren en op te ruimen, maar missen vaak native functionaliteit om direct gecomprimeerde `.csv.gz`-bestanden te query'en en zijn trager bij grote analytische aggregaties.

4. Relational OLTP (Server / Daemon-based):

- *Kenmerken:* De klassieke relationele databases (zoals PostgreSQL en MySQL) die werken met tabellen en relaties. Ze draaien altijd als een permanente achtergrondservice of Docker-container.
- *Geschiktheid voor use-case:* **Ongeschikt.** Het instellen en later weer opruimen van de server/container kost te veel tijd voor een snel experiment. Bovendien vereisen ze vaak extra tussenstappen (pipes/unzip) om gecomprimeerde CSV-bestanden te kunnen inladen via `COPY`.

5. Document Store (NoSQL):

- *Kenmerken:* Slaat gegevens op als semi-gestructureerde documenten (meestal JSON/BSON) in plaats van tabellen.
- *Geschiktheid voor use-case:* **Ongeschikt.** Vragen om een actieve database-service en zijn niet ontworpen voor relationele/analytische SQL-transformaties via dbt. Gecomprimeerde CSV-bestanden moeten eerst omgezet worden naar JSON.

6. Key-Value Store:

- *Kenmerken:* Slaat data op als simpele Sleutel-Waarde paren, veelal volledig in het werkgeheugen (RAM) voor extreem lage latency.
- *Geschiktheid voor use-case:* **Ongeschikt.** Niet gebouwd voor gestructureerde SQL-queries, tabel-transformaties of CSV-ingestie. Geen ondersteuning voor dbt.

7. Graph Database:

- *Kenmerken:* Richt zich op het opslaan van knooppunten (nodes) en relaties (edges) om complexe netwerken te analyseren.
- *Geschiktheid voor use-case:* **Ongeschikt.** Vereist zware runtime-omgevingen (vaak Java), ondersteunt geen standaard SQL/dbt-pijplijnen en maakt de verwerking van een platte CSV-dataset onnodig complex.

8. Multi-model Graph:

- *Kenmerken:* Combineert grafenstructuren met andere modellen (zoals document- of object-georiënteerd) binnen één database-engine.
- *Geschiktheid voor use-case:* **Ongeschikt.** Kampt met dezelfde nadelen als gewone graph databases: hoge overhead, zware installatie/service en gebrek aan dbt-integratie voor platte CSV-transformaties.

4.2 Conclusie: Welke architectuur past het beste?

Voor een snel CLI-experiment op een Linux-omgeving met `.csv.gz`-bestanden en dbt is de **In-process OLAP**-architectuur de overduidelijke winnaar.

- Het elimineert het beheer van achtergrondservices (zero-setup & zero-cleanup).
- Het kolomgeoriënteerde formaat verwerkt analytische transformaties vele malen sneller dan OLTP- of NoSQL-varianten.
- Het stelt de database in staat om gecomprimeerde CSV-bestanden direct in te lezen vanuit SQL.

5. Productvergelijking & Toolselectie (Library)

In dit hoofdstuk worden de specifieke database-tools binnen de meest geschikte architectuurcategorieën (*In-process OLAP* en *Columnar OLAP*) gedetailleerd vergeleken. De geselecteerde kandidaten worden getoetst aan de vooraf gestelde kaders om tot de definitieve keuze voor de Proof of Concept te komen.

5.1 Evaluatie van Kandidate Database-Engines

Op basis van de analyse in Hoofdstuk 4 zijn drie concrete database-technologieën geselecteerd die (deels) voldoen aan de eis voor een lokale, lichtgewicht en scriptbare verwerking:

1. **DuckDB** (*In-process OLAP*)
2. **ClickHouse Local** (*Columnar OLAP / In-process mode*)
3. **SQLite** (*In-process OLTP* ter vergelijking als de traditionele lichtgewicht referentie)

1. DuckDB

- **Architectuur:** In-process OLAP.
- **Native Linux & CLI:** Bevat een enkele, afhankelijkheidsvrije binary die direct via de CLI of vanuit scripts (Python, Bash) kan worden aangeroepen.
- **.csv.gz verwerking:** Biedt uitstekende native ondersteuning via de ingebouwde `read_csv_auto()` functie. Het gecomprimeerde bestand wordt direct vanuit SQL gelezen en verwerkt zonder vooraf uit te pakken op schijf.
- **dbt-Integratie:** Beschikt over een officieel en zeer actief onderhouden dbt-adapter (`dbt-duckdb`).

- **Installatie & Opruimen:** Direct te gebruiken zonder daemons. De gehele database bestaat uit een enkel bestand (bijv. `experiment.duckdb`) of kan zelfs *in-memory* (`:memory:`) worden gedraaid. Opruimen vereist enkel het verwijderen van dat ene bestand.

2. ClickHouse Local (`clickhouse local`)

- **Architectuur:** Columnar OLAP.
- **Native Linux & CLI:** Met `clickhouse local` biedt ClickHouse een standalone CLI-tool die werkt zonder dat er een server-daemon op de achtergrond hoeft te draaien.
- **.csv.gz verwerking:** Zeer snelle verwerking van gecomprimeerde bestanden via native SQL-table-functions (`file('data.csv.gz', 'CSV')`).
- **dbt-Integratie:** Beschikt over de `dbt-clickhouse` adapter, maar deze is voornamelijk ontworpen voor de client-server/cluster opzet van ClickHouse en vereist extra configuratie om soepel te integreren met de standalone CLI-modus op lokale bestanden.
- **Installatie & Opruimen:** De installatiefootprint (binary omvang) is aanzienlijk groter dan bij DuckDB, hoewel opruimen eveneens eenvoudig is als er gebruik wordt gemaakt van een lokaal bestand.

3. SQLite

- **Architectuur:** In-process OLTP.
- **Native Linux & CLI:** Uitstekende CLI- en script-ondersteuning, standaard aanwezig op vrijwel elk Linux-distributiesysteem.
- **.csv.gz verwerking:** Geen native ondersteuning voor `.csv.gz`. Gecomprimeerde bestanden moeten eerst op schijf worden uitgepakt of via ingewikkelde CLI-pipes/extensions worden ingeladen.
- **dbt-Integratie:** Ondersteund via `dbt-sqlite`, maar de transformaties op analytische queries zijn trager vanwege de rijgeoriënteerde opslag.
- **Installatie & Opruimen:** Zeer eenvoudig (enkel bestand), maar schiet tekort op het gebied van analytische performance en gecomprimeerde data-ingestie.

5.2 Matrix & Scorevergelijking op basis van Kaders

De kandidaten zijn getoetst aan de projectkaders en beoordeeld op een schaal van **1 tot 5** (*1 = Voldoet niet / Slecht, 5 = Uitstekend / Maximale geschiktheid*):

Criterium / Kader	DuckDB	ClickHouse Local	SQLite
1. Licentie / Open-Source	5 (<i>MIT</i>)	5 (<i>Apache 2.0</i>)	5 (<i>Public Domain</i>)
2. Native Linux & CLI	5 (<i>1 lichte binary</i>)	4 (<i>Zwaardere binary</i>)	5 (<i>Standaard op Linux</i>)

Criterium / Kader	DuckDB	ClickHouse Local	SQLite
3. Scriptable & Git-proof	5 (Eenvoudig via CLI/Python)	4 (Goede CLI, zwaardere setup)	5 (Eenvoudig via CLI/SQL)
4. Native .csv.gz Support	5 (Directe <code>read_csv_auto()</code>)	5 (Directe <code>file()</code> functie)	1 (Vereist unzippen op schijf)
5. dbt-Integratie	5 (Naadloze <code>dbt-duckdb</code>)	3 (Primair voor server-mode)	3 (Trage OLTP-transformaties)
6. Zero-Daemon Setup	5 (Pure in-process engine)	5 (In-process CLI-modus)	5 (Pure in-process engine)
7. Eenvoud van opruimen	5 (Wis 1 <code>.duckdb</code> bestand)	4 (Wis bestand/tijdelijke mappen)	5 (Wis 1 <code>.db</code> bestand)
8. Query Performance (OLAP)	5 (Vectorized Columnar)	5 (Vectorized Columnar)	2 (Rijgeoriënteerd/OLTP)
Eindscore (Totaal / 40)	40 / 40	35 / 40	31 / 40

5.3 Selectie & Conclusie

Op basis van de *Available Product Analysis* en de kwantitatieve scoring is **DuckDB** met de maximale score van **40/40** de meest geschikte database-oplossing voor deze use-case.

DuckDB onderscheidt zich op drie doorslaggevende punten van de andere open-source alternatieven:

- Efficiënte data-ingestie:** Het behaalt de maximale score (5/5) voor gecomprimeerde bestanden door `.csv.gz`-bestanden direct via SQL in te lezen zonder tussenkomst van unzipping-stappen op schijf.
- Volledige dbt-ondersteuning:** De `dbt-duckdb` adapter is specifiek gebouwd voor in-process verwerking, waar ClickHouse Local meer frictie geeft doordat de adapter voornamelijk op server-clusters is gericht.
- Maximale flexibiliteit & Nul rommel:** Doordat het als een enkele MIT-gelicenseerde binary draait zonder achtergrondservices, is zowel de opzet via scripts als het opruimen na afloop een kwestie van seconden.

Kritische noot: DuckDB is gebouwd voor single-writer gebruik, niet voor multi-user toegang. Voor deze casus is dat geen probleem, aangezien het experiment lokaal en single-user wordt uitgevoerd. Daarnaast kan de performance terugvallen zodra de dataset niet meer in het werkgeheugen past. Omdat in dit onderzoek gewerkt wordt met een sample van de volledige dataset, vormt ook dit geen belemmering voor de haalbaarheid van de PoC.

5.4 Evaluatie en Vergelijking van DuckDB CLI-Interfaces & Tooling

Nu **DuckDB** is geselecteerd als de meest geschikte *In-process OLAP* database-engine, dient bepaald te worden via welke specifieke Command Line Interface (CLI) of Terminal User Interface (TUI) de interactie met DuckDB het beste kan worden ingericht. Omdat DuckDB als bibliotheek/binary in diverse omgevingen kan worden aangeroepen, bestaan er verschillende manieren om de database in de terminal te bedienen.

5.4.1 Overzicht van de CLI-Kandidaten voor DuckDB

1. Officiële DuckDB CLI (`duckdb`)

- **Type:** Standalone Command Line Executable (C++ binary).
- **Kenmerken:** Gebaseerd op de klassieke SQLite-shell, maar sterk uitgebreid voor analytische workflows. Ondersteunt autocompletion, syntax highlighting, PostgreSQL-achtige commando's en ingebouwde `.mode` weergaven (zoals `csv`, `markdown`, `json` en `box`).
- **Geschiktheid voor scripten: Extreem hoog.** Kan direct vanuit Bash-scripts of CI/CD-pijplijnen worden aangeroepen via non-interactieve SQL-invocatie (bijv. `duckdb database.duckdb "SELECT ..."` of via STDIN pipes).

2. Harlequin TUI (`harlequin`)

- **Type:** Terminal User Interface (TUI) geschreven in Python (Textual framework).
- **Kenmerken:** Biedt een volwaardige grafische ervaring *binnen* de terminal. Bevat een schema-browser (om tabellen en kolommen in te zien), een multi-buffer SQL-editor met tabbladen en een scrollbare resultatenviewer.
- **Geschiktheid voor scripten: Laag (alleen voor interactief gebruik).** Harlequin is ontworpen voor interactieve gegevensverkenning (ad-hoc inspectie van tabellen) en is door de interactieve TUI-aard niet bedoeld voor geautomatiseerde batch-scripting of pipelines.

3. DuckDB Python CLI Executable / Scripting (`python -c` / `ipython`)

- **Type:** Taalspecifieke CLI-integratie via de Python `duckdb` module.
- **Kenmerken:** Maakt gebruik van de Python-bindings van DuckDB. Maakt het mogelijk om SQL-queries direct te combineren met Python-datastructuren, Polars/Pandas DataFrames of Arrow-tabellen.
- **Geschiktheid voor scripten: Zeer hoog.** Biedt maximale flexibiliteit voor complexere ingestie- en ETL-logica die verder gaat dan pure SQL, terwijl het nog steeds volledig scriptbaar is in Linux.

4. USQL (`usql`)

- **Type:** Universele SQL CLI-client (geschreven in Go).

- **Kenmerken:** Een uniforme command-line interface die met een breed scala aan databases kan praten (DuckDB, PostgreSQL, MySQL, SQLite, Oracle).
- **Geschiktheid voor scripten: Goed.** Biedt een consistente interface als binnen het project meerdere databasetypen naast elkaar worden gebruikt, maar voegt voor een pure DuckDB-workflow een extra externe afhankelijkheid toe.

5.4.2 Matrix Vergelijking DuckDB Tooling

De verschillende interfaces zijn getoetst aan de relevante kaders uit Hoofdstuk 2 en gewaardeerd op een schaal van **1 tot 5** (1 = *Niet geschikt / Slecht*, 5 = *Uitstekend / Maximale geschiktheid*):

Criterium / Kader	Officiële CLI (duckdb)	Harlequin TUI	Python CLI / Scripting
1. Automatisering & Scripting	5 (<i>Uitstekend in Bash/CI</i>)	1 (<i>Alleen interactief</i>)	5 (<i>Extreem flexibel</i>)
2. Git-versiebeheer & Reproduceerbaarheid	5 (<i>Eenvoudige <code>.sql</code> bestanden</i>)	3 (<i>Slaat alleen sessies op</i>)	5 (<i>Standaard <code>.py</code> scripts</i>)
3. Geen extra afhankelijkheden (Zero-Setup)	5 (<i>1 enkele afhankelijkheidsvrije binary</i>)	2 (<i>Vereist Python/pipx/Textual</i>)	3 (<i>Vereist Python-omgeving</i>)
4. Interactiviteit & Dataverkenning	3 (<i>Basis SQL-shell</i>)	5 (<i>Rijke TUI met schema browser</i>)	3 (<i>REPL/IPython afhankelijk</i>)
5. Uitvoerformaten (.csv, .json, markdown)	5 (<i>Ingebouwde <code>.mode</code> opties</i>)	3 (<i>Exporteert naar scherm/vijandig</i>)	5 (<i>Via DataFrames/Export</i>)
Eindscore (Totaal / 25)	23 / 25	14 / 25	21 / 25

5.4.3 Conclusie & Advies voor Inrichting

Uit de vergelijking blijkt dat er een duidelijk onderscheid gemaakt moet worden tussen geautomatiseerde verwerking (scripts/pijplijnen) en interactieve ad-hoc inspectie:

1. Primaire keuze voor automatisering & ingestie: Officiële DuckDB CLI (`duckdb`)

- Met een score van **23/25** is de officiële binary de aangewezen tool voor het uitvoeren van ingestie-scripts, het aanroepen van `.sql`-bestanden en de dbt-integratie. Het vereist geen runtime-afhankelijkheden (zoals Python of Go) en kan direct in een Linux-environment of Git-repository gebruikt worden.

2. Secundaire keuze voor interactieve dataverkenning (Optioneel): Harlequin TUI

- Voor de ontwikkelaar die tijdens het experimenteren visueel door het DuckDB-schema wil navigeren of snel resultaten van queries wil inspecteren in de terminal, vormt Harlequin de ideale *interactieve aanvulling*, zonder dat het de automatisering van de pijplijn in de weg zit.

6. Conclusie & Advies

6.1 Conclusie

Het doel van dit onderzoek was het bepalen van de meest geschikte, lokale database-oplossing voor het snel en geautomatiseerd experimenteren met gecomprimeerde datasets (.csv.gz) binnen een Linux-omgeving en met ondersteuning voor dbt.

Op basis van het DOT Framework (Available Product Analysis) is geconcludeerd dat:

1. **Architectuurkeuze:** De **In-process OLAP**-architectuur de beste aansluiting biedt op de kaders. Deze architectuur vereist geen achtergrondservice (zero-daemon setup), biedt extreme query-performance op analytische SQL-transformaties en maakt direct inlezen van gecomprimeerde data mogelijk.
2. **Database-engine:** **DuckDB** als absolute winnaar uit de productvergelijking komt met een maximale score van 40/40. DuckDB leest gecomprimeerde `.csv.gz`-bestanden native in via `read_csv_auto()`, beschikt over een officiële en volwaardige `dbt-duckdb` adapter, en vereist na afloop slechts het verwijderen van één enkel `.duckdb`-bestand om het systeem volledig schoon achter te laten.
3. **CLI-Tooling:** De **officiële DuckDB CLI** de primaire interface vormt voor de geautomatiseerde ingestie en de koppeling met Git/dbt (score 23/25), doordat deze afhankelijkheidsvrij is. Voor optionele, interactieve verkenning in de terminal kan **Harlequin TUI** als secundaire tool gebruikt worden.

6.2 Advies & Vervolgstappen

Op basis van deze resultaten wordt geadviseerd om de overige twee onderdelen van het DOT Framework **Prototyping** en **Integration Test**, zoals geïntroduceerd in Hoofdstuk 3 als volgt in te richten:

1. **Prototyping (Lab):**
Schrijf een Bash- of Python-script dat via de officiële DuckDB CLI het lokale `.csv.gz`-bestand direct inleest in een DuckDB-databasebestand (bijv. `experiment.duckdb`), en verifieer dat dit herhaalbaar is via Git-gecontroleerde scripts.
2. **Integration Test (Workshop / Lab):**
Richt een `profiles.yml` in op basis van de `dbt-duckdb` adapter die verwijst naar `experiment.duckdb` en valideer de keten door een basistransformatie (`dbt run`) uit te voeren.
3. **Versiebeheer:**
Plaats alle inrichtingsscripts en de dbt-projectstructuur in Git.

Bronnen

Google. (2026). Gemini 1.5 Pro (Versie van 22 september 2026) [Groot taalmodel].

<https://gemini.google.com>

(Noot: Ingezet voor het structureren van de vergelijkingsmatrices, synthese van database-architecturen en redactionele ondersteuning volgens de DOT-methodiek).

HBO-i. (2020). ICT research methods: Framework and guidelines for research-oriented software engineering. HBO-i Foundation. Geraadpleegd op 20 september 2026, van

<https://ictresearchmethods.nl/>

Verantwoording Inzet Generatieve AI & Co-creatie

Tijdens het tot stand komen van dit onderzoeksrapport is gebruikgemaakt van de generatieve AI-assistent Google Gemini (Gemini 1.5 Pro). De AI is gedurende het gehele traject ingezet als interactieve sparringpartner voor de synthese van de literatuur, de aanscherping van de onderzoeksstructuur en het opstellen van de kwantitatieve evaluatiematrices.

Het onderzoeksproces en de samenwerking zijn in de volgende stappen verlopen:

1. Aanscherping Kaders & Licentiecontrole:

In de beginfase is de selectie van database-kandidaten getoetst aan de vooraf gestelde projectkaders. Hierbij is specifiek ingezoomd op de licentievormen om te garanderen dat alle voorgestelde technologieën 100% open-source en vrij te gebruiken zijn binnen het project.

2. Verdieping in Database-architecturen:

Om de keus voor In-process OLAP theoretisch te onderbouwen, is samen met de AI geanalyseerd waarom vectorized columnar storage en SIMD-instructies op de CPU een kritische performance-winst opleveren bij het verwerken van gecomprimeerde datasets (.csv.gz).

3. Matrixvorming en Objectieve Scoring:

Op basis van de gekozen kaders (Linux-native, scriptable, dbt-integratie, zero-daemon) is het vergelijkend onderzoek omgezet naar een kwantitatieve matrix. De criteria zijn per rij voorzien van een transparante score (schaal 1-5) om de keus voor DuckDB (40/40 punten) objectief te onderbouwen.

4. Evaluatie van de Tooling-laag (CLI & TUI):

Na de selectie van DuckDB als database-engine is de scope verbreed naar de interactie-interface. De verschillende CLI- en TUI-mogelijkheden (zoals de officiële DuckDB CLI, Harlequin TUI en Python-scripting) zijn vergeleken om een helder onderscheid te maken tussen geautomatiseerde pipeline-ingestie en interactieve dataverkenning.

5. Synthese en Afsluiting:

Tot slot is het verloop van het gehele onderzoek samengebracht in een afsluitend

hoofdstuk (Conclusie & Advies) en is het rapport gestructureerd volgens de richtlijnen van het DOT Framework.

Verantwoording & Regie:

De inhoudelijke regie, het formuleren van de specifieke projectkaders, het aanleveren van de casuscontext en de uiteindelijke besluitvorming lagen volledig bij de onderzoeker. De AI is uitsluitend gebruikt om de onderzoeksresultaten te structureren, te verifiëren en redactioneel aan te scherpen.